



EDUQA.PE

INTRODUCCIÓN A SQLITE CON PYTHON



SESIÓN 02

Insertar, leer y confirmar

AUTOR

Alejandro Valentino Seminario Medina

AI Engineer



Insertar filas

La sentencia **INSERT INTO** recibe la lista de columnas y, tras la palabra **VALUES**, los valores en ese mismo orden.

Documentación: **INSERT**

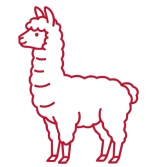


Los valores nunca van dentro de la cadena

Esta es la regla más importante del módulo. El signo de interrogación (?) actúa como marcador de posición, y los valores se pasan aparte en una tupla.

Nota técnica. Concatenar el valor dentro de la cadena SQL abre la puerta a una inyección: un valor como `\'" OR 1=1 --\'` cambia el significado de la consulta. Con marcadores de posición eso no puede ocurrir, porque el valor viaja por un canal distinto y nunca se analiza como SQL.

```
cursor.execute(  
    "INSERT INTO vendedores (nombre, region) VALUES (?, ?)",  
    ("Ana Quispe", "Norte"),  
)
```



Los valores nunca van dentro de la cadena

La tupla de un solo elemento necesita una coma final. Sin ella, Python no la interpreta como tupla.

Nota técnica. Escribir ("Norte") entrega una cadena, no una tupla, y el módulo lanza ValueError porque intenta usar cada carácter como un parámetro distinto. La coma final, ("Norte",), es lo que crea la tupla.

```
cursor.execute(  
    "SELECT * FROM vendedores WHERE region = ?",  
    ("Norte", ),  
)
```

Documentación: [Marcadores de posición](#) · [INSERT](#)



Marcadores con nombre

En lugar del signo de interrogación puede usarse dos puntos seguidos de un nombre (:nombre), y pasar un diccionario. Con muchas columnas resulta más legible.

```
cursor.execute(
    "INSERT INTO vendedores (nombre, region) VALUES (:nombre, :region)",
    {"nombre": "Luis Mendoza", "region": "Sur"},
)
```

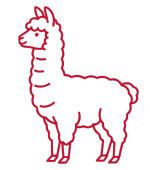
Documentación: [Marcadores de posición](#)



Insertar muchas filas

El método `executemany()` repite la misma sentencia sobre cada elemento de una secuencia, en una sola llamada.

Nota técnica. `executemany()` es mucho más rápido que un bucle con `execute()`: el motor analiza la sentencia una sola vez y confirma una sola transacción, en lugar de una por fila.



Insertar muchas filas

```
vendedores = [  
    ("Rosa Ttito", "Norte"),  
    ("Carlos Paz", "Centro"),  
]  
cursor.executemany(  
    "INSERT INTO vendedores (nombre, region) VALUES (?, ?)",  
    vendedores,  
)
```

Documentación: [Cursor.executemany\(\)](#)



Confirmar los cambios

Las escrituras quedan dentro de una transacción abierta hasta llamar a `commit()`. Sin esa llamada, se descartan al cerrar la conexión.

Nota técnica. El módulo abre una transacción de forma implícita antes de la primera sentencia que modifica datos. Las consultas de solo lectura no la necesitan, y por eso `SELECT` funciona sin `commit()`.

```
conexion.commit()
```

Documentación: [Connection.commit\(\)](#) · [Control de transacciones](#)



La conexión como gestor de contexto

Usar la conexión con la palabra **with** confirma la transacción al salir del bloque sin error, y la revierte si se lanza una excepción.

Nota técnica. El bloque **with** sobre la conexión gestiona la transacción, no la conexión en sí: al salir no cierra el archivo. Cerrarlo sigue siendo responsabilidad de `close()`.

```
with conexion:
    conexion.execute(
        "INSERT INTO vendedores (nombre, region) VALUES (?, ?)",
        ("Pedro Rojas", "Sur"),
    )
```



La conexión como gestor de contexto

Documentación: [La conexión como gestor de contexto](#)



Leer resultados

El método `fetchall()` devuelve todas las filas restantes como una lista de tuplas.

```
cursor.execute("SELECT nombre, region FROM vendedores")
print(cursor.fetchall())
```

Salida

```
[('Ana Quispe', 'Norte'), ('Luis Mendoza', 'Sur')]
```

El método `fetchone()` devuelve una sola fila, o `None` cuando ya no quedan.

Nota técnica. `fetchall()` carga todo el resultado en memoria. Con tablas grandes conviene recorrer el cursor directamente en un bucle `for`, porque entrega las filas de a poco.



Leer resultados

```
cursor.execute("SELECT nombre FROM vendedores")
print(cursor.fetchone())
print(cursor.fetchone())
```

Salida

```
('Ana Quispe',)
('Luis Mendoza',)
```

El cursor también es iterable, y esa es la forma recomendada para resultados grandes.

```
for fila in cursor.execute("SELECT nombre FROM vendedores"):
    print(fila[0])
```



Leer resultados

Salida

Ana Quispe

Luis Mendoza

Rosa Ttito

Documentación: [Cursor.fetchall\(\)](#) · [Cursor.fetchone\(\)](#) · [Cursor](#)



Acceder a las columnas por nombre

Por omisión cada fila es una tupla, y las columnas se leen por posición. Asignar `sqlite3.Row` al atributo `row_factory` permite leerlas por nombre.

Nota técnica. `sqlite3.Row` admite acceso por índice y por nombre, distingue mayúsculas de minúsculas solo en el índice, y ocupa menos memoria que un diccionario. Leer por posición hace que la consulta se rompa en silencio si alguien reordena las columnas.



Acceder a las columnas por nombre

```
conexion.row_factory = sqlite3.Row
cursor = conexion.cursor()

cursor.execute("SELECT nombre, region FROM vendedores")
fila = cursor.fetchone()
print(fila["nombre"], fila["region"])
```

Salida

Ana Quispe Norte

Documentación: [Connection.row_factory](#) · [sqlite3.Row](#)

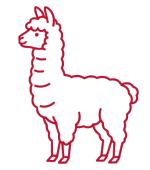


Modificar y borrar

La sentencia **UPDATE** cambia valores de filas existentes. La cláusula **WHERE** decide a cuáles se aplica.

Nota técnica. Sin cláusula WHERE, el UPDATE afecta a todas las filas de la tabla y nadie pide confirmación. El atributo cursor.rowcount indica cuántas filas cambió la última sentencia.

```
cursor.execute(
    "UPDATE vendedores SET region = ? WHERE nombre = ?",
    ("Oriente", "Carlos Paz"),
)
print(cursor.rowcount)
```



Modificar y borrar

Salida

1

La sentencia **DELETE FROM** elimina filas. La misma advertencia sobre la cláusula **WHERE** aplica.

```
cursor.execute("DELETE FROM vendedores WHERE region = ?", ("Oriente",))
conexion.commit()
```

Documentación: [UPDATE](#) · [Cursor](#) · [DELETE](#)



Cerrar la conexión

El método `close()` libera el archivo. Los cambios sin confirmar se pierden.

```
conexion.close()
```

Documentación: [Connection.close\(\)](#)



Fin de la sesión

Insertar, leer y confirmar

Alejandro Valentino Seminario Medina